

The 30-Day AI Builder Program

Open this every morning. Each day gives you timed blocks — what to study, exactly where to find it, the key points to focus on, and what to build — adding up to 4–5 hours of real work. Tick boxes as you go and keep a short log. By Day 30 you’ll have built a working AI app and understood how it works.

Phase 1
Foundations

Days 1–7

Phase 2
Understanding LLMs

Days 8–15

Phase 3
Building the Real Thing

Days 16–25

Phase 4
Polish, Evaluate & Present

Days 26–30

Daily commitment: 4–5 focused hours | Capstone: a Document Q&A Assistant (RAG)

The checkboxes and log boxes are fillable on screen — or print it and tick by hand.

How to Use This Workbook

Read the day, then work the blocks in order

Each day is split into timed blocks — usually Study, then Build, then Wrap. The minutes are a guide that adds up to 4–5 hours. Don't skip Study to rush to Build; the building only works if the ideas are in place.

'Where to go' is exact on purpose

Each block lists the specific page, chapter, video, or section to use — follow those links rather than searching loosely. The 'focus' checkboxes under each block are the key points: tick them as you cover each one.

Produce the one deliverable

Every day ends in a single deliverable (the green box). That's the proof you did the day. Tick it when it's genuinely done and committed to Git.

Keep the daily log

Ten minutes at the end of each day in the log box: what you did, what clicked, what you got stuck on. It's the fastest way for your supervisor to help you on the right thing tomorrow.

When you're stuck

Give it an honest attempt, check the linked materials, then ask a specific question — 'I expected X but got Y' beats 'it doesn't work.' Being stuck is normal and expected; hiding it is the only real mistake.

Checkpoints (Days 7, 15, 25, 30)

These end each phase. Don't rush past them — they're where you consolidate before the next phase builds on top of your work.

PHASE 1

Foundations

Days 1–7

Remove the magic. AI is data, statistics, and pattern-matching — you'll build the core vocabulary and workflow of machine learning.

DAY 1 Python for Data Work

~4 hrs

GOAL

Get fluent with the tools every AI workflow runs on — notebooks and the core data libraries. No machine learning yet, just confidence handling data.

SETUP · ~30 min Get your environment running

WHERE TO GO

- [Google Colab](#) — just open it, no install
- (Optional, local) [Anaconda Distribution](#)

FOCUS — TICK AS YOU COVER EACH

- Decide: use Google Colab (recommended — zero setup, runs in the browser) or install Python + Jupyter locally.
- Open a fresh notebook and run `print("hello")` in a cell to confirm it works.
- Create a folder for this internship and run ``git init`` in it (you'll commit work here all month).

STUDY · ~90 min Learn NumPy and Pandas basics

WHERE TO GO

- [pandas: "10 Minutes to pandas"](#) (read all sections, run each snippet)
- [NumPy: "NumPy quickstart"](#) (sections 'The Basics' through 'Indexing, Slicing')

FOCUS — TICK AS YOU COVER EACH

- NumPy — key points: what an array is, array shape, indexing/slicing, and element-wise operations.
- pandas — key points: what a DataFrame and a Series are; reading a CSV; selecting columns; filtering rows with a condition.
- pandas — the four commands you'll use constantly: `.head()`, `.info()`, `.describe()`, `.value_counts()`. Run each one.
- Don't just read — retype every snippet into your own notebook and run it. Reading without running teaches almost nothing.

BUILD · ~90 min Explore a real dataset yourself

WHERE TO GO

- [Kaggle: Titanic dataset](#) (download `train.csv`)
- Or load Iris directly: `from sklearn.datasets import load_iris`

FOCUS — TICK AS YOU COVER EACH

- Load the dataset into a DataFrame and look at it with `.head()` and `.info()`.
- Write down 5 questions you can answer from the data (e.g. "What % of passengers survived?", "Average age by class?").
- Answer each of your 5 questions with a line or two of pandas code (try `.groupby()` and `.mean()`).
- Add a one-line comment above each answer explaining what the code does.

WRAP · ~20 min Save and log

FOCUS — TICK AS YOU COVER EACH

- Tidy the notebook so it reads top-to-bottom and runs without errors (Runtime → Restart and run all).
- Commit it to your Git folder.
- Fill in today's log: what clicked, what felt confusing.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A notebook that loads a dataset and prints answers to 5 questions you posed, each with a one-line explanatory comment. Runs cleanly top-to-bottom.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 2 Core ML Concepts

~4 hrs

GOAL

Build the vocabulary of machine learning before training anything. Understand what “learning” actually means here, so later days aren’t just copy-paste.

STUDY · ~75 min The big picture of how ML works

WHERE TO GO

- [3Blue1Brown: “But what is a neural network?”](#) (~19 min video)
- [Google ML Crash Course](#) → module “Linear Regression” (intro + ‘Loss’ sections)

FOCUS — TICK AS YOU COVER EACH

Key idea: a model is a function with adjustable numbers (parameters); “training” means tuning those numbers to fit examples.

Key idea: “loss” measures how wrong the model is; training tries to make loss small.

Watch the 3B1B video for intuition; don’t worry about the math details yet — focus on the shape of the idea.

STUDY · ~75 min Supervised vs unsupervised, and overfitting

WHERE TO GO

- [Google ML Crash Course](#) → “Datasets, Generalization, and Overfitting” module
- [Google for Developers: ML terminology](#) (framing)

FOCUS — TICK AS YOU COVER EACH

Supervised learning = learning from labeled examples (input → known answer). Unsupervised = finding structure with no labels.

Key terms to nail: feature, label, training set, test set, model, prediction. Write each in your own words.

Overfitting = memorizing the training data instead of learning the pattern; it’s why we hold out a test set.

Analogy to remember: overfitting is a student who memorizes past exam answers but fails new questions.

BUILD · ~60 min See the workflow run end-to-end

WHERE TO GO

- [scikit-learn: “An introduction to machine learning” tutorial](#)

FOCUS — TICK AS YOU COVER EACH

Run the tutorial’s example that loads a dataset, fits a model, and predicts — even before you understand every line.

Just observe the shape of it: load data → create model → .fit() → .predict(). You’ll repeat this pattern all month.

Note any line you don’t understand in your log — you’ll revisit tomorrow when you build your own.

WRAP · ~30 min Write your glossary

FOCUS — TICK AS YOU COVER EACH

Write a one-page glossary, in your own words, of: supervised learning, unsupervised learning, feature, label, training data, test data, overfitting.

Read it aloud — if a definition doesn’t make sense spoken, it’s not clear enough yet.

Log the 2 terms you feel least sure about.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A one-page glossary in your own words defining: supervised learning, unsupervised learning, feature, label, training data, test data, overfitting.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

GOAL

Train a real classifier from start to finish. The point is owning the workflow — not the accuracy number.

STUDY · ~45 min How train/test splitting works

WHERE TO GO

- [Kaggle Learn: “Intro to Machine Learning” lessons 1–4 \(free, ~45 min\)](#)
- [scikit-learn: train_test_split reference](#)

FOCUS — TICK AS YOU COVER EACH

Key point: you train on one slice of data and test on a different, unseen slice — never the same rows.

Why: testing on data the model already saw tells you nothing about how it handles new cases.

Learn the one-liner: `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)`.

BUILD · ~2.5 hrs Train and run your own classifier

WHERE TO GO

- [scikit-learn: Logistic Regression user guide](#)
- [scikit-learn: DecisionTreeClassifier \(alternative model\)](#)

FOCUS — TICK AS YOU COVER EACH

Pick a dataset: Titanic survival (from Day 1) or a simple spam/not-spam set.

Separate your features (X) from your label (y), then split into train and test sets.

Create a model (`LogisticRegression()` or `DecisionTreeClassifier()`), call `.fit(X_train, y_train)`.

Call `.predict(X_test)` and print a handful of predictions next to the true answers.

If you hit errors about text or missing values, that's normal — search the exact error message; debugging is the skill.

WRAP · ~45 min Explain what you built

FOCUS — TICK AS YOU COVER EACH

Add a markdown cell answering, in plain English: what are my features? what is my label? what did the model learn?

Sanity check: did you accidentally train on the test set? You should not have.

Commit the notebook and log what was hardest.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A notebook that trains a classifier and outputs predictions, plus a markdown cell answering: what are my features, my label, and what is the model doing?

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

GOAL

Understand why “accuracy” alone can lie, and learn the metrics that tell you if a model is actually good.

STUDY · ~90 min Accuracy, precision, recall, confusion matrix

WHERE TO GO

- [Google ML Crash Course → “Classification” module \(sections: Thresholding, Confusion matrix, Accuracy, Precision and recall\)](#)
- [scikit-learn: confusion_matrix reference](#)

FOCUS — TICK AS YOU COVER EACH

Confusion matrix: understand the four cells — true positive, false positive, true negative, false negative.

Precision = of the things I flagged positive, how many really were? Recall = of all real positives, how many did I catch?

Key example to internalize: a test that always says ‘healthy’ can be 99% accurate yet catch zero sick patients.

Decide which matters more for a spam filter — precision or recall — and why.

BUILD · ~90 min Measure yesterday's model

WHERE TO GO

- [scikit-learn: classification_report \(gives all three metrics at once\)](#)

FOCUS — TICK AS YOU COVER EACH

Compute accuracy, precision, and recall on your Day 3 model's test predictions.
Print and read the confusion matrix; map each number back to what it means.
Now break it on purpose: retrain with a tiny training set (e.g. 10 rows) and watch the metrics drop.
Write one sentence explaining what changed and why.

WRAP · ~30 min **Write the 'accuracy lies' note**

FOCUS — TICK AS YOU COVER EACH

Write a short note describing a realistic scenario where a high-accuracy model is still bad.
Commit your evaluation notebook and log your takeaway.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A notebook section showing the confusion matrix and the three metrics, plus a written explanation of a scenario where high accuracy would still be a bad model.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 5 **Turning Text into Numbers — Part 1**

~4 hrs

GOAL

Understand how machines turn language into numbers — the single most important idea behind everything in Phases 2–3.

STUDY · ~90 min **Tokenization and embeddings, intuitively**

WHERE TO GO

→ [Jay Alammar: "The Illustrated Word2Vec" \(read in full — it's visual and beginner-friendly\)](#)
→ [Hugging Face NLP Course](#) → Chapter 2, "Tokenizers" section

FOCUS — TICK AS YOU COVER EACH

Tokenization = splitting text into pieces (words or sub-words) that the model handles one at a time.
Embedding = representing each token as a list of numbers (a vector) that captures meaning.
The big intuition from Word2Vec: words with similar meaning end up with similar vectors (close together in space).
The famous example: king – man + woman ≈ queen. Understand why that's remarkable.

BUILD · ~90 min **Get real tokens and vectors**

WHERE TO GO

→ [Google ML Crash Course](#) → "Embeddings" module (intro sections)
→ [sentence-transformers: Quickstart \(pip install, then encode\(\)\)](#)

FOCUS — TICK AS YOU COVER EACH

Tokenize a few sentences in code and print the tokens — notice how punctuation and sub-words are handled.
Use a small embedding model to get the vector for 5 words; print each vector's length (its dimension count).
Note in your log: every word became a list of the same number of floats. That fixed-length list is the embedding.

WRAP · ~30 min **Explain it back**

FOCUS — TICK AS YOU COVER EACH

Write a short note: what is a token, what is an embedding, and why would 'cat' and 'kitten' sit near each other?
Log anything about the math that felt fuzzy — you don't need the math, just the intuition.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A notebook that tokenizes sentences and retrieves embedding vectors for 5 words, plus a note on what the vector's dimensions represent conceptually.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

GOAL

Make embeddings tangible — measure similarity and see related words cluster. This is the direct bridge to semantic search in Phase 2.

STUDY · ~45 min Cosine similarity

WHERE TO GO

→ [scikit-learn: cosine_similarity reference + example](#)

→ [TensorFlow Embedding Projector \(play with it live\)](#)

FOCUS — TICK AS YOU COVER EACH

Cosine similarity = how aligned two vectors are, from -1 (opposite) to 1 (identical direction).

Why cosine and not plain distance: it compares direction (meaning) rather than magnitude.

Play with the Embedding Projector: search a word, see its nearest neighbors, get a feel for clusters.

BUILD · ~2.5 hrs Build a tiny similarity explorer

WHERE TO GO

→ [sentence-transformers: Computing embeddings + similarity](#)

→ [matplotlib: scatter plot basics \(for the 2D plot\)](#)

FOCUS — TICK AS YOU COVER EACH

Embed ~20 words across 2–3 categories (e.g. animals, countries, foods).

Compute cosine similarity for 5 word pairs; print a small table. Do related pairs score higher? They should.

Reduce the vectors to 2D (use PCA from sklearn) and make a scatter plot, labeling each point.

Confirm visually that same-category words clump together. If a plotting snippet blocks you, grab one and move on — the learning is in reading the plot.

WRAP · ~30 min Interpret the picture

FOCUS — TICK AS YOU COVER EACH

Write 2–3 sentences interpreting your plot: which clusters formed, any surprises?

Commit the notebook and the plot image; log your takeaway.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A 2D scatter plot of word embeddings showing visible clusters, plus a similarity table for 5 word pairs and a short interpretation.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 7 Phase 1 Wrap-Up

~4 hrs

CHECKPOINT

GOAL

Consolidate Phase 1. Putting concepts in your own words is how you discover what you truly understand — and what you don't yet.

BUILD · ~2 hrs Write “How Does a Machine Learn?”

WHERE TO GO

→ Re-skim your Day 2 and Day 4 notes and glossary

FOCUS — TICK AS YOU COVER EACH

Write a 1–2 page explainer titled “How Does a Machine Learn?” aimed at a smart friend with no ML background.

Cover the full arc: data → features & labels → training → testing → evaluation → why overfitting matters.

Work in the embeddings idea: how text becomes numbers a model can use.

Use your own analogies, not the tutorials' — reusing their words hides gaps in understanding.

BUILD · ~75 min Polish your Week 1 notebook

FOCUS — TICK AS YOU COVER EACH

Clean and comment your Day 3–4 classifier notebook so it reads as a finished piece of work.

Make sure it runs end-to-end with Restart and run all.

Add a short markdown intro at the top explaining what the notebook does.

WRAP · ~45 min Self-assess and prep to present

FOCUS — TICK AS YOU COVER EACH

Prepare a 5-minute informal walkthrough of what you built and learned this week.

List the 2 concepts you feel weakest on — be honest; this guides your supervisor's help.

Commit everything and log your reflection on Week 1.

DELIVERABLE — TICK WHEN DONE & COMMITTED

(1) A 1–2 page explainer “How Does a Machine Learn?”; (2) a cleaned, commented classifier notebook; (3) your list of 2 weakest areas.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

PHASE 2

Understanding LLMs

Days 8–15

Demystify large language models, work with them through an API, learn prompt engineering, and make embeddings practical with vector search.

DAY 8 What Is an LLM?

~4 hrs

GOAL

Demystify large language models. By tonight you should see them as sophisticated next-token predictors — not magic, not a search engine, not sentient.

STUDY · ~2 hrs How LLMs actually work

WHERE TO GO

- 3Blue1Brown: “Large Language Models explained briefly”, then “But what is a GPT?”
- Google ML Crash Course → “Large Language Models” module (tokens → Transformers intro)
- Jay Alammar: “The Illustrated GPT-2” (skim the visuals)

FOCUS — TICK AS YOU COVER EACH

Core mechanism: an LLM repeatedly predicts the next token given everything before it. That’s the whole engine.
Context window = how much text it can ‘see’ at once; once you exceed it, earlier text falls out of view.
Hallucination = it generates plausible-sounding text, not verified facts — so it can be confidently wrong.
Temperature = a knob for randomness: low = focused/repetitive, high = creative/risky.
Connect back to Phase 1: tokens become embeddings inside the model — same idea you already learned.

BUILD · ~75 min Probe the behavior (no API yet)

WHERE TO GO

- Anthropic: Claude Docs — Intro / overview
- Use any chat LLM you have access to, to run the experiments below

FOCUS — TICK AS YOU COVER EACH

Ask the same question 3 times and note variability — you’re seeing sampling/temperature in action.
Ask it something obscure and verifiable, then fact-check it — try to catch a hallucination.
Paste a very long block of text and ask about the first line — probe the context-window idea.

WRAP · ~30 min Write your LLM explainer

FOCUS — TICK AS YOU COVER EACH

Write a one-page note in your own words: next-token prediction, context window, hallucination, temperature.
Log the one thing that surprised you most today.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A one-page note explaining, in your own words: next-token prediction, context window, hallucination, and temperature.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 9 First API Calls

~4 hrs

GOAL

Make your first programmatic call to an LLM and understand the request/response shape you’ll use for the rest of the program.

SETUP · ~45 min Get API access safely

WHERE TO GO

- [Anthropic: Get started / Quickstart](#)
- (Whichever provider your supervisor chose — confirm before starting)

FOCUS — TICK AS YOU COVER EACH

Get your API key from your supervisor or the provider console.
Store the key in an environment variable (e.g. a .env file) — never paste it into code, never commit it to Git.
Add .env to your .gitignore right now, before writing any code.

STUDY · ~45 min Understand the message format

WHERE TO GO

- [Anthropic: Messages API reference](#) (read 'Request' and 'Examples')

FOCUS — TICK AS YOU COVER EACH

Three roles: system (instructions/persona), user (your input), assistant (the model's reply).
A call sends a list of messages and gets back the assistant's message.
Note the key parameters: model, max_tokens, messages, and (optionally) system.

BUILD · ~2 hrs Your first scripts

WHERE TO GO

- [Anthropic: Quickstart code sample](#)

FOCUS — TICK AS YOU COVER EACH

Write a 'hello world' script: send one user message, print the reply.
Add a system prompt that gives the model a persona; observe how the tone changes.
Loop: send 3 different prompts and print all 3 replies in one run.
Read the key out of the environment variable, not a hardcoded string — confirm no key is in your file.

WRAP · ~20 min Commit cleanly

FOCUS — TICK AS YOU COVER EACH

Double-check git status / your diff to be 100% sure no API key is being committed.
Commit the script and log anything confusing about the API.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A working Python script that sends a prompt to an LLM API and prints the response, committed to your repo with no API key in the code.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 10 Practical LLM Tasks

~4.5 hrs

GOAL

Use the LLM for real text tasks and build honest intuition for where it shines and where it fails.

STUDY · ~30 min What LLMs are good at

WHERE TO GO

- [Anthropic: Prompt engineering overview](#)

FOCUS — TICK AS YOU COVER EACH

Skim the overview to see the categories of task LLMs handle well (summarizing, classifying, extracting, rewriting).
Note: clear instructions + relevant context = better output. You'll prove this to yourself today.

BUILD · ~3 hrs Build three small tools

WHERE TO GO

- [Anthropic: Increase output consistency \(for the extraction task\)](#)

FOCUS — TICK AS YOU COVER EACH

Tool 1 — Summarizer: paste in an article, get a 3-bullet summary.
Tool 2 — Sentiment classifier: given text, return positive / negative / neutral.
Tool 3 — Extractor: pull all names and dates from a paragraph into a clean list.

Test each on at least 3 different inputs — not one cherry-picked example.
Deliberately feed each tool a tricky input (ambiguous sentiment, a paragraph with no dates) and see what breaks.

WRAP · ~30 min Document strengths and weaknesses

FOCUS — TICK AS YOU COVER EACH

For each tool, write one strength and one weakness you observed.
Commit all three scripts and log which task the model handled worst.

DELIVERABLE — TICK WHEN DONE & COMMITTED

Three small scripts (summarize, classify, extract) plus a note listing one strength and one weakness observed in each.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 11 Prompt Engineering — Part 1

~4 hrs

GOAL

Learn that how you ask changes what you get. Move from random prompts to deliberate, repeatable ones.

STUDY · ~90 min Core prompting techniques

WHERE TO GO

- Anthropic: "Be clear, direct, and detailed"
- Anthropic: "Give Claude a role (system prompts)"
- Anthropic: "Use examples (multishot prompting)"

FOCUS — TICK AS YOU COVER EACH

The golden rule: if a confused colleague couldn't follow your prompt, the model can't either.
Three levers today: clear/specific instructions, a system-prompt role, and giving relevant context.
Multishot = include 3–5 examples of input→output in the prompt; it steers format and style powerfully.
Examples should be relevant and varied so the model doesn't latch onto an accidental pattern.

BUILD · ~2 hrs Iterate a prompt and add examples

WHERE TO GO

- Reuse your Day 10 sentiment classifier

FOCUS — TICK AS YOU COVER EACH

Start from a deliberately weak prompt and improve it across 4+ versions, saving each version and its output.
After each change, write one line: what did this change improve?
Convert your sentiment classifier to few-shot by adding 3–5 labeled examples; compare against the zero-shot version.
Notice which kinds of inputs the examples fixed.

WRAP · ~30 min Capture what worked

FOCUS — TICK AS YOU COVER EACH

Write a short summary of which prompt change gave the biggest jump in quality.
Commit your prompt-iteration document and log your takeaway.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A document showing one prompt evolving across 4+ versions, with output at each stage and a note on what each change improved.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

GOAL

Get reliable, structured output — the thing that makes prompts usable inside real software.

STUDY · ~75 min Structured output and step-by-step reasoning

WHERE TO GO

- Anthropic: "Increase output consistency" (JSON / format control)
- Anthropic: "Let Claude think (chain of thought)"
- Anthropic: "Use XML tags"

FOCUS — TICK AS YOU COVER EACH

To get JSON: explicitly say 'respond with only valid JSON, no other text' and describe the exact fields.
 Chain-of-thought = ask the model to reason step by step; it improves hard/multi-step answers (but costs tokens).
 XML tags help the model separate instructions from data — wrap inputs in clear tags.
 Reality check: models sometimes return malformed JSON, so your code must handle that gracefully.

BUILD · ~2 hrs Build a prompt library + a JSON-reliable script

WHERE TO GO

- Python: `json.loads` / `try-except` for safe parsing

FOCUS — TICK AS YOU COVER EACH

Create a prompt-library file with 3–4 reusable, documented prompt templates (summarize, classify, extract, etc.).
 Write a script that asks for JSON and parses it with `json.loads` inside a `try/except`.
 When parsing fails, retry or print a clear error — don't let the program crash.
 Test with 5 inputs and confirm it never crashes, even on a bad response.

WRAP · ~45 min Tidy and reflect

FOCUS — TICK AS YOU COVER EACH

Organize your prompt library so it's easy to import and reuse in Phase 3.
 Commit and log: when is chain-of-thought worth the extra tokens, and when is it overkill?

DELIVERABLE — TICK WHEN DONE & COMMITTED

A prompt-library file (3–4 documented, reusable templates) and a script that reliably gets and parses JSON, handling malformed output without crashing.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 13 Embeddings & Vector Search — Part 1

~4.5 hrs

GOAL

Turn Phase 1's embeddings into a real power: finding relevant text by meaning, not keywords. This is the heart of the capstone.

STUDY · ~75 min Semantic search vs keyword search

WHERE TO GO

- Pinecone: "What is similarity search / semantic search"
- Anthropic: Embeddings guide

FOCUS — TICK AS YOU COVER EACH

Keyword search matches exact words; semantic search matches meaning (so 'car' can match 'automobile').
 The recipe: embed your documents once, embed the query, return the documents whose vectors are most similar.
 This is exactly cosine similarity from Day 6, now applied to whole sentences/passages.

BUILD · ~2.5 hrs Build a semantic search by hand

WHERE TO GO

- `sentence-transformers: encode()` + `util.cos_sim`

FOCUS — TICK AS YOU COVER EACH

Collect ~20 short docs/sentences (e.g. FAQ-style snippets).

Embed all of them and store the vectors in a Python list alongside the original text.
For a query, embed it and compute similarity against every doc; return the top 3.
Build a tiny keyword search too (substring match) and compare results on a query with synonyms.
Find one query where semantic search clearly wins — that's your proof it works.

WRAP · ~30 min Document the win

FOCUS — TICK AS YOU COVER EACH

Write a short note comparing semantic vs keyword results on your example query.
Commit and log how this connects to the Day 16 RAG idea (it's the 'retrieve' step).

DELIVERABLE — TICK WHEN DONE & COMMITTED

A script that takes a query and returns the 3 most semantically similar documents, plus a note comparing it to keyword matching.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 14 Embeddings & Vector Search — Part 2

~4 hrs

GOAL

Move from a hand-rolled list to a real vector database, and understand why one is needed once you have real data volumes.

STUDY · ~45 min Why a vector database

WHERE TO GO

→ [Chroma: Getting Started](#)
→ [\(Alt\) FAISS: Getting started wiki](#)

FOCUS — TICK AS YOU COVER EACH

A vector DB stores embeddings and finds nearest neighbors fast — even across millions of items.
Your Day 13 Python loop works for 20 docs but would crawl at 200,000. The DB solves that.
Chroma is the gentlest to start with: collection → add documents → query.

BUILD · ~2.5 hrs Reimplement search on Chroma

WHERE TO GO

→ [Chroma: Usage guide \(add / query / metadata\)](#)

FOCUS — TICK AS YOU COVER EACH

Install Chroma, create a collection, and add your ~20 docs (Chroma can embed them for you).
Run the same query as Day 13 and confirm you get comparable top results.
Attach a metadata field (e.g. a source name) to each doc — you'll rely on this for citations later.
Make the script re-runnable without creating duplicates.

WRAP · ~30 min Lock it in

FOCUS — TICK AS YOU COVER EACH

Write one paragraph: what does the vector DB do that your Python list couldn't?
Commit and log any setup gotchas for future-you.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A script using Chroma (or FAISS) that ingests documents with metadata and answers similarity queries, replacing the manual approach from Day 13.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

GOAL

Combine your LLM skills into one working tool, and make sure you can explain the whole pipeline — because Phase 3 is assembling exactly these pieces.

BUILD · ~2.5 hrs Build a CLI question-answering tool

WHERE TO GO

→ Reuse your Day 9 API script and Day 12 prompt library

FOCUS — TICK AS YOU COVER EACH

Build a command-line tool: user types a question, the tool sends it to the LLM and prints the answer.

Use a system prompt and clean formatting; handle empty or weird input without crashing.

Keep the code tidy — you may reuse parts of it in the capstone.

BUILD · ~60 min Compare prompts + organize code

FOCUS — TICK AS YOU COVER EACH

Run 2–3 different system prompts through your tool and record how answer quality changed.

Organize Phase 2 code into a tidy folder (api, prompts, search) and commit.

WRAP · ~30 min Explain the full path

FOCUS — TICK AS YOU COVER EACH

In your log, explain end-to-end: text → tokens → embeddings → similarity → LLM answer.

If any step is fuzzy, flag it — fix it before Phase 3 starts.

DELIVERABLE — TICK WHEN DONE & COMMITTED

(1) A working CLI question-answering tool; (2) a written prompt-comparison note; (3) organized Phase 2 code in your repo.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

PHASE 3

Building the Real Thing

Days 16–25

Assemble everything into a working Retrieval-Augmented Generation (RAG) application with a usable interface.

DAY 16 RAG Architecture — Part 1

~4 hrs

GOAL

Understand the Retrieval-Augmented Generation pattern conceptually before writing a line of capstone code. If you can draw it, you can build it.

STUDY · ~2 hrs What RAG is and why it exists

WHERE TO GO

- [Pinecone: "Retrieval Augmented Generation" \(read in full\)](#)
- [Anthropic: "Contextual retrieval" \(skim for how retrieval quality matters\)](#)

FOCUS — TICK AS YOU COVER EACH

The flow: question → retrieve relevant chunks from your docs → put chunks + question in the prompt → grounded answer.

Why RAG: it lets the LLM answer about documents it was never trained on, and citing sources cuts hallucination.

Key insight: the LLM never 'knows' your documents — you hand it the relevant pieces at question time.

Each RAG component maps to something you already built: embeddings (Day 5–6), vector search (Day 13–14), prompting (Day 11–12).

BUILD · ~90 min Diagram the pipeline yourself

WHERE TO GO

- [Use pen and paper, draw.io, or Excalidraw](#)

FOCUS — TICK AS YOU COVER EACH

Draw the full RAG pipeline as labeled boxes and arrows: ingestion (one-time) and query (every question).

Under each box, write which Phase 1–2 skill it uses.

Write one paragraph: why does RAG beat just asking the LLM the question directly?

WRAP · ~30 min Teach it back

FOCUS — TICK AS YOU COVER EACH

Explain your diagram out loud (to the other intern, or a rubber duck) without looking at notes.

Log the one part of the pipeline you understand least — that's what to watch on build days.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A clear, labeled architecture diagram of the RAG pipeline, plus a paragraph on why RAG beats asking the LLM directly.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 17 Project Setup

~4 hrs

GOAL

Scaffold the capstone cleanly so the rest of the phase is just filling in well-organized pieces — not fighting a tangled mess.

STUDY · ~45 min How to structure a Python project

WHERE TO GO

- [Real Python: "Python Application Layouts" \(read the 'application' sections\)](#)

FOCUS — TICK AS YOU COVER EACH

- Separate concerns into folders: ingestion, retrieval, generation, ui.
- Use a requirements.txt (or pyproject) so the project is reproducible.
- Keep config (model name, chunk size) in one place, not scattered through the code.

BUILD · ~2.5 hrs Scaffold the repo and pick your docs

WHERE TO GO

→ Your supervisor will confirm the document set — aim for something real and useful

FOCUS — TICK AS YOU COVER EACH

- Confirm the document corpus your assistant will answer over (a real knowledge base beats toy data).
- Create the folder structure and empty module files with a one-line comment of each file's job.
- Write a README stub: what the project does, the document set, and your planned architecture (paste the Day 16 diagram).
- List the components to build, in order, as a checklist in the README.

WRAP · ~30 min Commit the skeleton

FOCUS — TICK AS YOU COVER EACH

- Commit the scaffold so you have a clean starting point.
- Log: is the scope realistic for the days remaining? Note any worry now.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A scaffolded repo with a sensible folder structure, a README describing the project and document set, and an ordered checklist of components to build.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 18 Document Ingestion — Part 1

~4.5 hrs

GOAL

Build the pipeline that loads raw documents and splits them into good chunks — the single biggest lever on final answer quality.

STUDY · ~75 min Why and how to chunk

WHERE TO GO

- [Pinecone: "Chunking strategies" \(read in full\)](#)
- [LangChain: "Text splitters" concept page](#)

FOCUS — TICK AS YOU COVER EACH

- Why chunk: whole documents are too big for the context window and hurt retrieval precision.
- Chunk size is a trade-off: too big = noisy retrieval, too small = lost context. Start around a few hundred tokens.
- Overlap: let chunks share a little text at the edges so ideas spanning a boundary aren't cut in half.
- Good chunks respect structure (paragraphs/sentences) rather than slicing mid-word.

BUILD · ~2.5 hrs Load and chunk your documents

WHERE TO GO

- [pypdf \(if loading PDFs\)](#)
- [LangChain RecursiveCharacterTextSplitter \(optional helper\)](#)

FOCUS — TICK AS YOU COVER EACH

- Write a loader for your document type (start with .txt or .pdf).
- Implement chunking with a fixed size plus overlap (write it yourself or use a splitter).
- Print 5 sample chunks and eyeball them: are they coherent? not too long? not mid-sentence fragments?
- Tune the size/overlap until the chunks look sensible.

WRAP · ~30 min Sanity check

FOCUS — TICK AS YOU COVER EACH

- Note your chosen chunk size and overlap, and why, in the README.
- Commit the ingestion-part-1 script and log how the chunks look.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A script that loads the document set and splits it into clean, inspectable chunks (with a sensible size and overlap).

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 19 Document Ingestion — Part 2

~4 hrs

GOAL

Finish ingestion: embed every chunk and store it (with its source) in the vector database so it's ready to retrieve.

STUDY · ~30 min Storing chunks with metadata

WHERE TO GO

- [Chroma: Usage guide — add documents + metadata](#)
- [Anthropic: Embeddings](#)

FOCUS — TICK AS YOU COVER EACH

Each stored item = the chunk text + its embedding + metadata (source file, maybe page or section).
Metadata is what lets you cite sources on Day 21 — don't skip it now, it's painful to add later.

BUILD · ~2.5 hrs Embed and store everything

WHERE TO GO

- Reuse your Day 14 Chroma setup

FOCUS — TICK AS YOU COVER EACH

Loop over your Day 18 chunks: embed each and add it to the vector DB collection.
Attach source metadata to every chunk as you add it.
Run a test query and inspect what comes back — confirm the text and its source both return.
Make the script re-runnable (clear or upsert) so re-ingesting doesn't create duplicates.

WRAP · ~60 min Verify the store

FOCUS — TICK AS YOU COVER EACH

Query 3 different topics and confirm relevant chunks come back with correct sources.
Commit the complete ingestion pipeline and log any indexing issues.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A complete ingestion script: documents in → chunked, embedded, and stored in the vector DB, with source metadata on every chunk.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 20 Retrieval + Generation — Part 1

~4.5 hrs

GOAL

Wire retrieval to the LLM — the moment your system answers its first real question from your documents.

STUDY · ~45 min The retrieve-then-generate prompt

WHERE TO GO

- [Pinecone: RAG \(re-read the 'generation' part\)](#)
- [Anthropic: Prompt engineering overview \(context placement\)](#)

FOCUS — TICK AS YOU COVER EACH

The query-time flow: embed the question → retrieve top-k chunks → build a prompt that includes those chunks → ask the LLM.
Put the retrieved chunks clearly in the prompt (e.g. inside XML tags) and instruct: answer using only this context.

'k' is how many chunks you retrieve — start with 3–5.

BUILD · ~3 hrs Connect the pieces end-to-end

WHERE TO GO

→ Reuse Day 19 retrieval + Day 9 API call

FOCUS — TICK AS YOU COVER EACH

Write a function: take a question, retrieve the top-k chunks from the vector DB.

Build a prompt string that embeds those chunks plus the question.

Send it to the LLM and print the grounded answer.

Test with several real questions about your documents and read the answers critically.

Trace one question through every step and confirm you understand each hop.

WRAP · ~30 min First milestone

FOCUS — TICK AS YOU COVER EACH

It works end-to-end now — commit this milestone.

Try one question your documents can't answer; note what it does (you'll fix this tomorrow).

DELIVERABLE — TICK WHEN DONE & COMMITTED

A script that answers a typed question using retrieved document context — your first end-to-end working version of the assistant.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 21 Grounding & Citations

~4 hrs

GOAL

Make answers trustworthy: grounded in the documents, showing their sources, and honest enough to say "I don't know."

STUDY · ~45 min Reducing hallucination

WHERE TO GO

→ Anthropic: "Reduce hallucinations"

FOCUS — TICK AS YOU COVER EACH

Tell the model explicitly: answer only from the provided context; if it's not there, say you don't know.

Ask the model to cite which chunk each claim came from — give the chunks IDs or labels it can reference.

Honest refusal is a feature, not a failure — a system that makes things up is worse than one that admits gaps.

BUILD · ~2.5 hrs Add citations and 'I don't know'

WHERE TO GO

→ Reuse your Day 20 pipeline

FOCUS — TICK AS YOU COVER EACH

Label each retrieved chunk with its source, and instruct the model to cite the source it used.

Add the instruction to reply 'I don't know based on the documents' when the context lacks the answer.

Test refusal with 3 out-of-scope questions — it should decline, not invent.

Display the cited sources alongside each answer.

WRAP · ~45 min Verify honesty

FOCUS — TICK AS YOU COVER EACH

Confirm citations point to the chunk the answer actually came from.

Commit and log: did it still hallucinate anywhere? Note where.

DELIVERABLE — TICK WHEN DONE & COMMITTED

An upgraded assistant that returns answers with source citations and correctly refuses questions outside the document set.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 22 Building a UI — Part 1

~4.5 hrs

GOAL

Wrap your working pipeline in a simple web interface so it feels like a product, not a script. Function first.

STUDY · ~60 min Learn Streamlit basics

WHERE TO GO

- [Streamlit: "Get started" + "Main concepts"](#)
- (Alt) [Gradio: Quickstart](#)

FOCUS — TICK AS YOU COVER EACH

Streamlit turns a Python script into a web app — you write normal Python, it renders the UI.
Learn the few widgets you need: `st.text_input`, `st.button`, `st.write`, `st.spinner`.
Run an app with ``streamlit run app.py``; it opens in your browser and reloads on save.

BUILD · ~3 hrs Put a UI on your assistant

WHERE TO GO

- [Streamlit: API reference \(look up widgets as needed\)](#)

FOCUS — TICK AS YOU COVER EACH

Build a minimal page: a text box for the question and an area that shows the answer.
Import your Day 21 pipeline and call it when the user submits — don't copy the logic, reuse it.
Get a real question answered through the browser, end-to-end.
Resist styling for now — make it work first, make it pretty on Day 23.

WRAP · ~30 min Commit the app

FOCUS — TICK AS YOU COVER EACH

Commit the working UI.
Log anything awkward about wiring the UI to your pipeline.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A locally running web app where a user types a question and sees the assistant's answer.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 23 Building a UI — Part 2

~4 hrs

GOAL

Polish the interface into something genuinely usable and demoable — and test it on a real person.

BUILD · ~2.5 hrs Add the niceties

WHERE TO GO

- [Streamlit: API reference \(spinner, expander, columns\)](#)

FOCUS — TICK AS YOU COVER EACH

Show the cited sources under each answer (an expander works well).
Add a loading spinner while the model thinks, and clear/disable the input appropriately.
Handle empty input gracefully — no blank questions sent to the model.
Add a small panel listing which documents are currently loaded.

BUILD · ~60 min Real usability test

WHERE TO GO

- Hand it to the other intern (or your supervisor) with no instructions

FOCUS — TICK AS YOU COVER EACH

Watch someone use it cold — do not explain anything.
Write down every moment of confusion or hesitation.
Fix the top 2 confusions immediately.

WRAP · ~30 min Lock the UI

FOCUS — TICK AS YOU COVER EACH

Commit the polished app.
Log what the usability test taught you.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A polished, usable web app showing answers with citations, loading states, and graceful handling of empty input.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 24 Testing & Edge Cases — Part 1

~4 hrs

GOAL

Probe your system's limits and find where it breaks. Start thinking like an engineer who breaks things on purpose.

STUDY · ~30 min How to test an LLM system

WHERE TO GO

→ [Anthropic: "Create strong empirical evaluations" / develop tests](#)

FOCUS — TICK AS YOU COVER EACH

Good testing is adversarial: you're trying to make it fail, not confirming it works.
When something fails, first ask: was the right chunk even retrieved? Most RAG failures are retrieval failures, not the LLM.

BUILD · ~3 hrs Run a structured test pass

FOCUS — TICK AS YOU COVER EACH

Brainstorm 10+ tricky inputs: ambiguous questions, out-of-scope, very long, empty, nonsense, multi-part.
Run each and record input, expected behavior, actual behavior, pass/fail in a table.
For each failure, label the likely cause: retrieval, prompt, chunking, or a crash.
Pick the top 2–3 problems worth fixing tomorrow.

WRAP · ~30 min Prioritize fixes

FOCUS — TICK AS YOU COVER EACH

Commit your test log.
Log your top fix priorities for Day 25.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A test-log table — input, expected behavior, actual behavior, pass/fail — for at least 10 varied inputs, with a likely cause noted per failure.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 25 Phase 3 Wrap-Up

~4 hrs

CHECKPOINT

GOAL

Fix the most important issues and lock a stable v1. After today you have a working product; Phase 4 only makes it better.

BUILD · ~2.5 hrs Fix the top issues

WHERE TO GO

→ Your Day 24 test log

FOCUS — TICK AS YOU COVER EACH

Fix the top 2–3 problems you identified — change one thing at a time.

After each fix, re-run the relevant tests to be sure you didn't break something else.

If a fix is bigger than today allows, note it as a known limitation instead of half-doing it.

BUILD · ~60 min Re-test and tag v1

WHERE TO GO

→ [Git: Basics of tagging](#)

FOCUS — TICK AS YOU COVER EACH

Re-run your full Day 24 test log and confirm the fixes held.

Commit and tag a stable 'v1' you can always return to.

WRAP · ~30 min Document limitations

FOCUS — TICK AS YOU COVER EACH

Write a short, honest list of known remaining limitations.

Log how it feels to have a working product — and what you're proudest of so far.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A stable, committed v1 of the assistant, with the test log re-run and a documented list of known limitations.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

PHASE 4

Polish, Evaluate & Present

Days 26–30

Turn your prototype into something you're proud of: tuned, measured, documented, extended, and presented.

DAY 26 Improving Answer Quality

~4 hrs

GOAL

Measurably improve your assistant through deliberate, single-variable tuning — not random tweaks you can't attribute.

STUDY · ~45 min The levers you can pull

WHERE TO GO

- [Pinecone: Chunking strategies \(revisit with fresh eyes\)](#)
- [Anthropic: Prompt engineering overview](#)

FOCUS — TICK AS YOU COVER EACH

Three main levers: chunk size/overlap, number of chunks retrieved (k), and the generation prompt.
The discipline: change ONE lever at a time so you know what caused any change.
Decide a fixed set of test questions you'll judge every change against.

BUILD · ~2.75 hrs Run tuning experiments

FOCUS — TICK AS YOU COVER EACH

Lock a fixed set of ~8 questions as your judging set.
Run experiment 1 (e.g. change k from 3 to 5); compare answers before/after; keep or revert.
Run experiments 2 and 3 on the other levers, the same way.
Record each experiment: what changed, before/after answers, keep/revert decision.

WRAP · ~30 min Capture the gains

FOCUS — TICK AS YOU COVER EACH

Note which single change helped most.
Commit your tuned version and the experiment log.

DELIVERABLE — TICK WHEN DONE & COMMITTED

An experiment log showing at least 3 single-variable tuning changes, each with before/after answers and a keep/revert decision.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 27 Evaluation

~4.5 hrs

GOAL

Measure the system honestly with a real test set — moving from “it feels good” to “it scores X%.”

STUDY · ~45 min What a good eval set looks like

WHERE TO GO

- [Anthropic: “Create strong empirical evaluations”](#)

FOCUS — TICK AS YOU COVER EACH

A test set = questions paired with known-correct answers your documents actually support.
Cover a range: easy lookups, multi-part questions, and a few out-of-scope (correct answer = ‘I don't know’).
Grade honestly — an honest 70% you can improve beats a flattering 95% that hides problems.

BUILD · ~3 hrs Build and run the eval**FOCUS — TICK AS YOU COVER EACH**

- Write 10–15 questions with known correct answers.
- Run every question through the assistant and record its answer.
- Score each correct / partial / wrong, and compute an overall accuracy.
- Group the failures — which kinds of question does it handle worst?

WRAP · ~30 min Write the eval report**FOCUS — TICK AS YOU COVER EACH**

- Write up the question set, the answers, the scores, the overall number, and the weakest categories.
- Commit the report and log what the score tells you.

DELIVERABLE — TICK WHEN DONE & COMMITTED

An evaluation report: the question set, the system's answers, the scores, an overall accuracy number, and a note on the weakest question types.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 28 Documentation

~4 hrs

GOAL

Write documentation good enough that a stranger could run your project from the README alone — a real, career-long skill.

STUDY · ~30 min What a great README contains**WHERE TO GO**

- [makeareadme.com](#) (structure + example)
- [GitHub: About READMEs](#)

FOCUS — TICK AS YOU COVER EACH

- A README answers: what it does, how to install it, how to run it, and its limits.
- Write for someone with zero context — spell out every setup step and command.

BUILD · ~2.75 hrs Write the documentation**FOCUS — TICK AS YOU COVER EACH**

- Write the full README: overview, setup, run instructions, usage example.
- Embed your architecture diagram (updated to match what you actually built).
- Add your Day 27 evaluation results and the known limitations.
- Add a 'what I'd do with more time' section.

WRAP · ~45 min Test the README literally**WHERE TO GO**

- Have the other intern follow your README from scratch

FOCUS — TICK AS YOU COVER EACH

- Watch someone set up the project using only your README; every stumble is a gap to fix.
- Fix the gaps, commit, and log what was unclear.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A complete, professional README with setup instructions, architecture diagram, evaluation results, limitations, and a future-work section — verified by someone else following it.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 29 Stretch Goal

~4.5 hrs

GOAL

Extend your project in one meaningful direction, mostly on your own — the best evidence of how far you've come.

STUDY · ~45 min Pick one and scope it

WHERE TO GO

→ [Conversation memory: Anthropic multi-turn messages](#)

→ [Deployment: Streamlit Community Cloud \(free\)](#)

FOCUS — TICK AS YOU COVER EACH

Choose exactly ONE: conversation memory (follow-ups), multi-file-type support, web deployment, or a second LLM provider.

Scope it to something you can finish today — a small working version beats a big unfinished one.

Read just enough of the relevant docs to start.

BUILD · ~3 hrs Build it yourself

FOCUS — TICK AS YOU COVER EACH

Implement your chosen feature, leaning on the docs and your own debugging.

Notice how much you can now figure out without step-by-step hand-holding — that's the growth.

Keep it working: don't break v1 to add the feature; commit once it works.

WRAP · ~30 min Document the addition

FOCUS — TICK AS YOU COVER EACH

Add a short section to the README describing the new feature and how to use it.

Log what you learned implementing it on your own.

DELIVERABLE — TICK WHEN DONE & COMMITTED

One working new feature, committed and documented, plus a note on what you learned implementing it independently.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

DAY 30 Final Presentation

~4 hrs

CHECKPOINT

GOAL

Bring the whole 30 days together: present your project and show what you understand — not just what you built.

BUILD · ~2 hrs Prepare the presentation

FOCUS — TICK AS YOU COVER EACH

Make 5–10 slides (or a tight demo script) covering: what it does, the architecture, your eval results, and key concepts learned.

Prepare a live demo: pick 3–4 questions that show it off, including one 'I don't know' to show honesty.

Rehearse once end-to-end and time yourself.

BUILD · ~75 min Present and demo

FOCUS — TICK AS YOU COVER EACH

Walk through the architecture using your diagram — explain each step in your own words.

Run the live demo.

Be ready for 'why' questions: why RAG instead of fine-tuning? what happens if retrieval fails?

WRAP · ~45 min Reflect

FOCUS — TICK AS YOU COVER EACH

Write a one-page reflection: what was hardest, what you're proudest of, what you'd learn next.

Make sure everything is committed and the README is current.

Final log entry — look back at Day 1 and note how far you've come.

DELIVERABLE — TICK WHEN DONE & COMMITTED

A final presentation + live demo, and a one-page written reflection on the 30 days.

MY DAILY LOG — WHAT I DID, WHAT CLICKED, WHAT I GOT STUCK ON

You Made It — What's Next

By Day 30 you have a real, demoable project and a working mental model of how modern AI systems are built: embeddings, retrieval, prompting, and honest evaluation. You've also built the habit of measuring instead of guessing — the thing that separates people who use AI from people who build with it.

Good directions to keep going

Deployment & sharing: put your assistant online so others can use it.

Evaluation at scale: bigger, automated test sets and tracking quality over time.

Agents & tools: let the assistant take actions (look things up, call APIs), not just answer.

Cost & latency: caching, smaller models for easy queries, and measuring your spend.